Few-shot classification

as attention over a support set

Implement a classifier that uses attention over labelled support examples to classify queries. Focus on tensor indexing, matrix operations and numerical stability.

01 Episode inputs

INPUT	SHAPE / TYPE	MEANING
`support_embeddings`	`float [N_s, D]`	Support feature vectors
`support_labels`	`int64 [N_s]`	Class IDs in [0, C)
`query_embeddings`	`float [N_q, D]`	Query feature vectors
`query_labels`	`int64 [N_q]`	Target class IDs in [0, C)
`num_classes`	`integer C`	Total number of classes
`temperature`	`float (default 0.1)`	Positive attention temperature

02 For every query

1. Compute cosine similarity to every support example.
2. Divide similarities by `temperature`.
3. Apply softmax over the support examples.
4. Obtain class probabilities by summing the attention weights of support examples belonging to each class.
5. Return the mean negative log-likelihood of the query labels and the predicted classes.

03 Implement

```
def matching_loss(
    support_embeddings,
    support_labels,
    query_embeddings,
    query_labels,
    num_classes,
    temperature=0.1,
):
    ...
```

Return: `(loss, predictions)` - a scalar mean NLL and `LongTensor[N_q]` of highest-probability class IDs. Keep the loss differentiable with respect to both embedding tensors.

Requirements. Do not assume examples are grouped by class or that classes have equal support counts. Use vectorized, numerically stable PyTorch operations. Treat a zero embedding as a zero normalized vector.

Assume valid inputs: non-empty sets, a shared positive embedding dimension, finite floating-point embeddings of the same dtype, tensors on one device, valid int64 labels, C > 0 and a finite positive temperature. Every query target class occurs in the support set; other classes may be absent and receive zero probability. Input validation is not required.

Be ready to explain your shapes, numerical choices, complexity and tests. No training or dataset download is needed.